CPSC 426
Assignment 2: Face Animation with Blend Shapes

Due 11:59PM, February 13, 2026

(a) Base (b) Smile (c) Yell (d) Sad (e) Confused

Figure 1: The base mesh and four predefined facial expressions.

1 Introduction

The goal of this assignment is to demonstrate understanding of blend shapes by implementing facial animation using Python with GPU acceleration via NVIDIA Warp. You will use a custom blend shape data format consisting of a base mesh and delta vectors, and use Polyscope [1], a real-time 3D mesh visualizer, to display and debug your animated results. You will work based on the starter code we have provided, which includes Python files (`.py` files in the `src/` folder).

Your task is to implement core blending and interpolation functionality that is missing from the Python files. The starter code offers hints and structure via `TODO` comments, which outline areas you will need to change. You may need to make open-ended design decisions requiring class and other refactoring outside of these `TODO` locations. Note that any `...` (ellipsis) indicate placeholders that require replacement or removal. You should reference concepts discussed in previous lectures on blend shapes, linear interpolation, and Catmull-Rom splines, and be able to explain the behavior of your system and design choices during F2F grading.

This assignment is organized into three parts, which are detailed in Section 2:

Part I: Blending. Implement GPU-accelerated blend shape deformation using Warp. You will complete the `blend_kernel` to compute blended vertex positions in parallel, set

up the `BlendState` class to manage delta and vertex position information, and implement an interactive slider interface in `make_slider_callback`. Starting from a base mesh with all blendshape weights at zero, you will create sliders that let you control each individual blendshape weight (0.0 to 1.0) and observe real-time facial deformations as weighted deltas are applied to the base mesh vertices.

Part II: Expression Morphing with Interpolation. Only upon **completing Part I**, you will implement time-based animation between predefined facial expressions using interpolation. First, you will complete `make_lerp_callback` to implement linear interpolation (LERP) between consecutive expressions, computing blendshape weights based on elapsed program time. Then, you will implement Catmull-Rom spline interpolation by completing `catmullrom_spline_weights` (computing the four basis function values) and `make_catmullrom_callback` (using four control points for smoother curves). Both interpolation methods create continuous animation cycles that smoothly loop through different facial expressions, with animation speed controlled by a UI slider.

1.1 Getting the Code

Assignment code is hosted on the course's GitHub repository:

https://github.students.cs.ubc.ca/CPSC426-2025W-T2/a2-release

Students registered in this course should be able to access the repo by logging in with their CWL ID. If you have problems with logging in, please head to the CS departmental account setup page (https://www.cs.ubc.ca/getacct/), log in with your CWL ID and check if your student account has been activated. And if you still have problems, please create a private post on Piazza so the teaching team can take a look into the issue.

1.2 Template

1.2.1 Source Files

- `main.py` is the entry point for the program. It loads the base mesh and blend shape delta data, initializes the `BlendState`, sets up Polyscope visualization, and registers interactive callbacks. You should look at this file, but you do not need to modify it.
- `blend.py` contains the `BlendState` class which manages blend shape data and GPU blending operations, as well as the `blend_kernel` Warp kernel for parallel vertex blending. You will implement the kernel, which relies on proper `BlendState` buffer initialization, and complete the update pipeline to apply blendshape deltas.
- `interpolate.py` contains callback functions for Polyscope's interactive UI. You will implement `make_slider_callback` for manual blendshape control (Part I), then `make_lerp_callback` and `make_catmullrom_callback` for linear and spline-based expression animation (Part II).

- `spline.py` contains the `catmullrom_spline_weights` function for computing the four Catmull-Rom spline basis function values given a parameter $u \in [0, 1)$. You will implement this function using the basis formulas provided in Sec. 2.2.
- `cli.py` handles command-line argument parsing for mode selection (`--mode`), device selection (`--device`), and profiling (`--profile`). You do not need to modify this file.
- `utils.py` contains helper functions for loading and formatting data. You do not need to modify this file.

1.2.2 Data

The `data/` directory contains the mesh and blend shape data:

- `base_mesh.obj` - A standard triangle `.obj` file containing the base or neutral mesh geometry, which we extract the vertices of in `utils.py`
- `base_deltas.txt` - Custom format defining individual blend shapes. Each line contains a blend shape name followed by delta vectors (dx, dy, dz) for each vertex.
- `expression_*_weights.txt` files - These are predefined "expressions" for facial expression configurations (e.g., `expression_smile_weights.txt`, `expression_yell_weights.txt`). Each expression is a weighted combination of several blendshapes defined in `base_deltas.txt`, and has a corresponding file that stores target weights for the blend shapes that are involved. Unmentioned blend shapes' weights are simply defaulted to 0.0 in main.

All file loading is already implemented in `main.py` and `utils.py`. The original source data comes from the Momentum Human Rig (MHR) [2] assets; specifically, the `lod3.fbx` file that contains the base mesh, a full-body rig, and a number of blendshapes (which we have converted to our custom text format for this assignment). You can import the `lod3.fbx` into Blender and modify blend shape weights in `Properties Side Panel` → `Data` → `Shape Keys` to compare your implementation to a ground truth.

1.3 Execution

Please read the `README.md`.

2 Work to be done (100 pts)

Be sure that your environment is set up correctly following the instructions in `README.md`. You should be able to run your code and visualize in Polyscope.

2.1 Part I: Blending (50 pts)

In this part, you will implement GPU-accelerated blend shape deformation and an interactive slider interface. Run with `--mode slider` to test.

(a) **20 pts** Complete the BlendState class.

In `blend.py`, complete the `BlendState` class:

- In `__init__`, create and initialize class variables as needed. You may benefit from looking at what is needed in part `(b)` for this, but at the very least you will need device buffers for base mesh vertices and vertex deltas.
- In `update`, copy blend weights to the device, launch `blend_kernel` (see below), copy blended vertices back to the host, and update the Polyscope mesh.

(b) **20 pts** Implement the Warp blending kernel.

In `blend.py`, complete the `blend_kernel` Warp kernel. Each thread computes the blended position for one vertex by accumulating weighted deltas on top of the base position:

$$\mathbf{v}_i = \mathbf{b}_i + \sum_{j=0}^{m-1} w_j \cdot \boldsymbol{\delta}_{j,i}$$

where $\mathbf{b}_i$ is the base vertex position, w_j is the weight for blend shape j, $\boldsymbol{\delta}_{j,i}$ is the delta for blend shape j at vertex i, and m is the number of blend shapes.

To complete the kernel, you will need to pass necessary information to the kernel and perform shape blending.

(c) **10 pts** Implement the slider callback.

In `interpolate.py`, complete `make_slider_callback`. For each blend shape and weight, you should create one Polyscope ImGui slider controlling blend shape weight (ranging from 0.0 to 1.0). If any weight changes during the callback check, call `blender.update()`. **Be sure to properly name these sliders**.

2.2 Part II: Expression Morphing with Interpolation (35 pts)

Important: Only begin Part II after completing Part I.
In this part, you will animate the face by interpolating between a set of predefined expressions over time. The expressions serve as key frames placed at equal intervals (see `seconds_per_segment` in `main.py`) along the timeline, with program time t moving the animation forward.

You will implement time-based animation between facial expressions using two interpolation methods. Each method computes a segment index and local parameter from elapsed time:

$$i = \lfloor t/s \rfloor \bmod n, \quad u = (t/s) - \lfloor t/s \rfloor, \quad u \in [0, 1)$$

where t is elapsed time, s is seconds per segment, n is the number of expressions, i is the index of the current segment (0-indexed), and u is the local interpolation parameter. Using i and u, you can find the collection of expressions relevant to segment i (the "neighborhood"), and compute the weights for all vertices using the neighborhood and interpolation parameter u.

By default, there are four expressions (five counting the base), which are defined in `main.py` to interpolate across.

(d) **10 pts** Implement linear interpolation (LERP).

In `interpolate.py`, complete `make_lerp_callback`. Given consecutive expression weight vectors $\mathbf{w}_i$ and $\mathbf{w}_{(i+1) \bmod n}$, compute:

$$\mathbf{w}(u) = (1-u) \cdot \mathbf{w}_i + u \cdot \mathbf{w}_{(i+1) \bmod n}$$

Pass the result to `blender.update()`. Run with `--mode lerp` to test.

(e) **10 pts** Implement Catmull-Rom spline basis weights.

In `spline.py`, complete `catmullrom_spline_weights(u)` to return the weights of a 4-point neighborhood of a Catmull-Rom spline, given interpolation parameter u, as a NumPy array. Recall that a Catmull-Rom spline has the following basis functions for its 4-point neighborhood:

$$\begin{aligned}
c_0(u) &= \tfrac{1}{2}(-u^3 + 2u^2 - u) \\
c_1(u) &= \tfrac{1}{2}(3u^3 - 5u^2 + 2) \\
c_2(u) &= \tfrac{1}{2}(-3u^3 + 4u^2 + u) \\
c_3(u) &= \tfrac{1}{2}(u^3 - u^2)
\end{aligned}$$

(f) **15 pts** Implement Catmull-Rom spline callback.

In `interpolate.py`, complete `make_catmullrom_callback`. Select four control points with wraparound:

$$\mathbf{p}_0 = \mathbf{w}_{(i-1) \bmod n}, \quad \mathbf{p}_1 = \mathbf{w}_i, \quad \mathbf{p}_2 = \mathbf{w}_{(i+1) \bmod n}, \quad \mathbf{p}_3 = \mathbf{w}_{(i+2) \bmod n}$$

Compute the interpolated weights using:

$$\mathbf{w}(u) = c_0 \cdot \mathbf{p}_0 + c_1 \cdot \mathbf{p}_1 + c_2 \cdot \mathbf{p}_2 + c_3 \cdot \mathbf{p}_3$$

The Catmull-Rom spline smoothly interpolates between $\mathbf{p}_1$ and $\mathbf{p}_2$ using $\mathbf{p}_0$ and $\mathbf{p}_3$ to control tangents. Run with `--mode cmrom` to test.

2.3 Part III: Feature Extension (15 pts)

Important: Starting from A2, we require students to separate the implementation of feature extension from the other parts for clearer grading. Please see Section 3.1 before you proceed.
For this part, your task is to extend the assignment to add features of your own choosing. The goal is to encourage you to unleash your creativity! The extension must be non-trivial to receive credit, but for full credit, the features have to be truly creative. See Sec. 4.2 for the grading expectations. A small number of exceptional extensions will be selected from this group and shown in class, with the student's permission. Some ideas (not all of which are the same difficulty!):

- **Easing Functions:** Add non-linear easing (ease-in, ease-out, ease-in-out) to the interpolation parameter u before computing weights. You may also make the time per segment non-uniform.
- **Additional Spline Methods:** Implement an alternative spline interpolation scheme and callback (e.g., B-spline, Hermite) and compare the smoothness and behavior to Catmull-Rom.
- **Audio-Driven Animation:** Drive blend shape weights from an audio source (e.g., microphone input or audio file) to create lip-sync or reactive facial animation.
- **Expression Blending UI:** Build a more advanced UI that lets you save slider configurations as named expressions.
- **GPU Interop:** Supposedly, there is a way to avoid doing a CPU to GPU copy of the blending weights each step. This may be impossible, but Polyscope documentation seems to support this with a "beta" preface. If so, do this and use `--profile` to report your findings before and after! Alternatively, you could try an OpenGL renderer (Warp has an OpenGL rendering option as well).

3 Submission Instructions

3.1 Directory Structure

Your submission should maintain the `src/` folder for the basic requirements, so that in F2F grading we can easily assign marks for the first two parts. For feature extensions, please make a `src2/` folder that if run from, shows your extension. If your feature extension requires more complex separation, you can tweak this. We only ask that you do not duplicate excessively (e.g., do not copy `data/` twice).

Please add your name, student number, and CWL username in the existing `README.md` file. You should modify the first line's template and replace it with your information. Also, you can include any information you would like to pass on to the marker below that (otherwise leaving N/A is fine).

3.2 Submission Methods

Please compress everything under the root directory of your assignment into an `a2.zip` and submit it on Canvas. You can make multiple submissions, but we will grade only the last one.

4 Grading

4.1 Face-to-face (F2F) Grading

To get a grade for the assignment, you must meet face-to-face with a TA in a ∼10-min slot on Zoom to demonstrate that you understand how your program works. Grading slots and detailed instructions on how to sign up and conduct F2F grading will be announced on Canvas and on Piazza.

While the exact process and questions may vary, you can generally expect the following during the meeting: The TA will (1) look at and/or run your code to inspect its correctness; (2) ask you to explain parts of your code, **including Python syntax and library features you use**; (3) ask you understanding-based questions about the assignment and logic. You are expected to answer them in a reasonable timeframe (based on difficulty).

The questions are intended to be mostly based on the code we ask you to implement, so blend shapes, interpolation methods, Warp concepts, and spline mathematics that you will have seen in lecture and while completing the assignment. We hope that the questions are more conversational than robotic; not everything we say is a test to deduct points from you. In most cases, any awkwardness or nervousness is just noise, so feel free to take pauses and ask for clarifications if anything confuses you. We may ask deeper questions of "why", and from that perspective it is okay to not immediately have an answer. We want to see your thought process in figuring out one.

4.2 Point Allocation

All questions before Feature Extension have a total of 85 points. The points are warranted based on

- The functional correctness of your program, i.e., how visually close your results are to expected results;
- The algorithmic correctness of your program, e.g., correctly implementing the blending kernel, interpolation methods, and spline basis functions;
- Your demonstration of understanding of the code through your answers to TAs' questions during face-to-face (F2F) grading.

The feature extension grade is based on creativity and sophistication of your submission. We warrant partial marks on four scales:

- 15: Outstandingly creative and sophisticated feature extension, e.g. the suggestions provided in Sec. 2.3. But note that only a very small number of extensions may get this score, and we cannot guarantee that if you implement one of the suggestions as is then you will definitely get full marks;
- 10: Average creativity and sophistication, e.g. adding easing functions, implementing an additional spline method;
- 5: Basic creativity and sophistication, e.g. uses the existing APIs to combine into a novel result;
- 0: Trivial or no feature extension, e.g. only changes CLI parameters, making purely cosmetic tweaks without new logic, or adding an extension that does not work reliably / cannot be explained during grading.

4.3 Collaboration Policy

Submissions must be individual, but you are encouraged to discuss the assignment with your classmates, and you are allowed to use generative AI tools, e.g. ChatGPT, Claude to help with constructing your solution.

In the beginning lines of the `README.md` file, include names of the people you discussed the assignment with, websites you acquired information from, and if you used generative AI tools, include the names of the tools you used.

You must understand your code and be able to explain it during F2F grading. If you cannot demonstrate at least a basic understanding of your code during face-to-face grading, you may receive a zero for the assignment—even if your code runs perfectly. Hiring a tutor to write your assignment, or copy-pasting others' code without proper referencing in `README.md` are considered plagiarism, and will be reported to the dean's office.

4.4 Penalties

Aside from penalties from incorrect solution or plagiarism, we may apply the following penalties to each assignment:
Late penalty. You are entitled up to three grace (calendar) days in total throughout the term. No penalties would be applied for using them. However, once you have used up the grace days, a deduction of 10 points would be applied to each extra late day. Note that

(a) The three grace days are given for all assignments, **not per assignment**, so please use them wisely;

(b) We check the time of your last submission to determine if you are late or not.

No-show penalty. Please sign up for a grading slot on the provided sign-up spreadsheet (link will be posted later on Piazza) before the submission deadline of the assignment, and show up to your slot on time. A 10-point deduction would be applied to each of the following circumstances:

(a) Not signing up for a grading slot before the sign-up period closes. Unless otherwise stated, the period closes at the same time as the submission deadline.

(b) Not showing up at your grading slot.

If none of the provided slots work for you, or if you have already missed your slot, follow instructions outlined in the given Piazza post for F2F grading to get graded after the grading period ends. Also, please note the following:

(a) You will need to explain to your TA why you're getting graded late, and may be asked to present documents to justify your hardship. The TA may remove the no-show penalty as long as he/she deems the justification to be reasonable.

(b) In the past some students reported that their names disappeared mysteriously due to technical glitches, and the spreadsheet's edit history has no trace of it. So, double-check that your name is on the sign-up sheet after you sign up by refreshing the page.

(c) Please do not try to modify other students' times or act in bad faith. Again, the sign-up sheet has history.

5 References

References

[1] Nicholas Sharp and others. *Polyscope*, 2019. https://polyscope.run/

[2] Aaron Ferguson et al. "MHR: Momentum Human Rig", 2025. arXiv:2511.15586. https://github.com/facebookresearch/MHR